

Atelier 10 — La Pagination en PHP (3 éléments par page)

Objectif de l'atelier

Dans cet atelier, nous allons apprendre à :

- Afficher les étudiants par pages
 - Limiter le nombre de résultats affichés
 - Utiliser `LIMIT` en SQL
 - Utiliser `$_GET`
 - Créer une pagination simple
-

1. Pourquoi utiliser la pagination ?

Imaginez une table contenant :

- 100 étudiants
- 500 produits
- 1000 utilisateurs

✗Afficher tous les résultats dans une seule page pose plusieurs problèmes :

- page lente
- mauvaise lisibilité
- surcharge de données

☑Solution : la pagination.

La pagination permet d'afficher :

- 3 étudiants par page
 - ou 10 produits par page
 - ou 20 utilisateurs par page
-

2. Principe de fonctionnement

Exemple avec 9 étudiants :

Page Étudiants affichés

Page 1 1 → 3

Page 2 4 → 6

Page 3 7 → 9

3. Structure SQL utilisée

Nous allons utiliser :

`LIMIT debut, nombre`

Exemple :

```
SELECT * FROM student LIMIT 0,3
```

Cela signifie :

- commencer à partir de l'index 0
 - afficher 3 résultats
-

4. Création de la méthode de pagination

Méthode complète

```
// GetStudentsPagination
static function GetStudentsPagination($page)
{
    // nombre d'étudiants par page
    $limit = 3;

    // calcul du premier élément
    $offset = ($page - 1) * $limit;

    // requête SQL
    $req = "SELECT * FROM student LIMIT $offset, $limit";

    // exécution
    $cur = Dataaccess::selection($req);

    return $cur;
}
```

5. Explication détaillée du code

Étape 1 — Nombre d'éléments par page

```
$limit = 3;
```

Ici nous décidons :

☒ afficher seulement 3 étudiants par page.

Étape 2 — Calcul du départ

```
$offset = ($page - 1) * $limit;
```

Cette formule est très importante.

Elle permet de calculer :

où commencer l'affichage.

6. Comprendre le calcul du offset

Page 1

```
$page = 1;
```

```
$offset = (1 - 1) * 3;  
$offset = 0;
```

SQL :

```
LIMIT 0,3
```

Résultat :

- étudiant 1
 - étudiant 2
 - étudiant 3
-

Page 2

```
$page = 2;
```

```
$offset = (2 - 1) * 3;  
$offset = 3;
```

SQL :

```
LIMIT 3,3
```

Résultat :

- étudiant 4
 - étudiant 5
 - étudiant 6
-

Page 3

```
$page = 3;
```

```
$offset = (3 - 1) * 3;  
$offset = 6;
```

SQL :

```
LIMIT 6,3
```

Résultat :

- étudiant 7
 - étudiant 8
 - étudiant 9
-

7. Récupération du numéro de page

Nous allons utiliser \$_GET.

Exemple URL

```
page.php?page=2
```

Code PHP

```
$page = 1;

if(isset($_GET['page']))
{
    $page = $_GET['page'];
}
```

8. Appel de la méthode

```
$students = Student::GetStudentsPagination($page);
```

Cette ligne récupère uniquement :

☒ les étudiants de la page demandée.

9. Création des liens de pagination

```
// GetTotalPages
static function GetTotalPages()
{
    // requête SQL
    $req = "SELECT COUNT(*) FROM student";

    // exécution de la requête
    $cur = Dataaccess::selection($req);

    // nombre total d'étudiants
    $totalStudents = 0;

    // lecture du curseur
    while($row = $cur->fetch())
    {
        $totalStudents = $row[0];
    }

    // nombre d'étudiants par page
    $limit = 3;

    // calcul du nombre total de pages
    $totalPages = ceil($totalStudents / $limit);

    return $totalPages;
}
```

```
<a href="?page=1">1</a>
<a href="?page=2">2</a>
<a href="?page=3">3</a>
```

10. Fonctionnement global

Si l'utilisateur clique sur :

```
?page=2
```

Alors :

```
$page = 2;
```

Puis :

```
$offset = 3;
```

Puis SQL exécute :

```
SELECT * FROM student LIMIT 3,3
```

11. Affichage des étudiants

```
foreach($students as $s)
{
    echo $s['nom']."<br>";
}
```

12. Schéma récapitulatif

```
Utilisateur clique sur page 2
    ↓
$_GET['page'] = 2
    ↓
Calcul du offset
    ↓
LIMIT 3,3
```

↓
Affichage des étudiants 4 → 6